

The Style Mimicker

Teaching a *frozen* model to write in David’s voice (no fine-tuning)

David Falk
Solanus Casey Archival Project

Abstract

This document explains the **Style-Mimickry** toolkit: a small system that makes a *frozen* large language model write in my voice and then *measures* whether it succeeded. Because I have forbidden fine-tuning, none of the style can live in the model’s weights; it must instead be *carried into* the model at request time (retrieved real examples of my writing) and *measured from outside* (an embedding “center of mass” of my prose plus an interpretable stylometry audit). The toolkit reuses the **step_7** libraries wholesale, so every model call is cost-logged and the model itself is a swappable configuration variable. Nothing here trains, downloads a large model, or spends money until explicitly asked.

Contents

1	The problem, and why “no fine-tuning” reshapes it	1
2	Data preparation: the real product	2
3	Generation in three layers	3
3.1	Layer 1 — retrieved exemplars + few-shot (runs today)	3
3.2	Layer 2 — a StyleVector-style nudge (scaffolded)	3
3.3	Layer 3 — Horikawa-style refinement (scaffolded and gated)	3
4	Evaluation: three honest measurements	4
5	Why this is honest about its limits	4
6	Caveats	5
	Sources	5

1 The problem, and why “no fine-tuning” reshapes it

I want a model to write the way I write. The usual answer is to fine-tune — to nudge the weights until the model’s default voice *becomes* mine. I have ruled that out. That single constraint is clarifying rather than merely limiting: if the style cannot live in the weights, then it has to live somewhere I can *see*, and the whole design becomes inspectable. There is no mysterious tensor that “knows” my voice; there is a folder of my actual sentences and a number that says how close a candidate is to them.

So I split the word “voice” into two jobs:

1. **Carry** the voice into the model at request time, by retrieving real examples of my writing and showing them to the model (few-shot prompting).
2. **Measure** the voice from the outside, with an embedding centroid of my prose and a plain-English stylometry dashboard.

One half makes the model *sound* like me; the other half tells me, honestly, whether it did. The three Python files map one-to-one onto this: `data_prep.py` builds the materials, `generate.py` carries the voice in, `evaluate.py` measures it.

A note on what we deliberately omit. The research report’s strongest recommendation is a fine-tuned hybrid (synthetic neutral→styled pairs, QLoRA, a reconstruction objective). All of that is gone, on purpose. What survives is exactly the subset that works on a frozen model: retrieved-exemplar prompting, a RAG index, a style centroid, a (prompt-proxy) StyleVector nudge, a gated Horikawa refinement loop, and the interpretable evaluation.

2 Data preparation: the real product

`data_prep.py` reads my writings — at present the `.tex` explainers in `examples/writing_examples` — and does three things.

1) Strip the scaffolding. My style lives in the sentences, not in `\usepackage{tikz}`. So before anything else we remove the LaTeX preamble, the `figure/tikzpicture/lstlisting` environments, inline math, and stray commands, leaving a prose approximation. This is best-effort and regex-only; it does not need a real LaTeX parser, because the goal is just to clean the *style signal*, not to render the document.

2) Segment into exemplars. We cut the prose into passages of roughly 40–180 words. That band is not arbitrary. The stylometry literature warns that count-based features are essentially noise below ~100 words and get diluted across too many ideas above ~180, so a paragraph-sized window is the sweet spot. Each exemplar keeps a pointer back to the file it came from, so anything I later show the model — or that the evaluator flags — can be traced to my actual writing.

3) Embed (only with `-embed`). The one paid step. We embed every exemplar with the `step_7` model-agnostic embedder, write them to a partitioned vector store under a `style::model@dim` space, and save the **style centroid**.

What the centroid is, intuitively. Imagine every passage I have written as a point in a high-dimensional space where “near” means “stylistically similar.” The centroid is the *center of mass* of that cloud — the single point that best summarizes “the average of my voice.” Because the embedder returns unit vectors, asking “how close is this new text to the centroid?” is just a cosine,

$$\cos(\mathbf{v}_{\text{text}}, \mathbf{c}) = \mathbf{v}_{\text{text}} \cdot \mathbf{c} \in [-1, 1],$$

where bigger means more like me. That one number is the target the refinement loop steers toward and the anchor the evaluator scores against.

```
python data_prep.py # FREE: segment + write exemplars.jsonl + profile
python data_prep.py --embed # PAID: also embed -> RAG index + style_centroid.npy
```

3 Generation in three layers

The layers are ordered cheapest-and-most-reliable first, exactly as the research recommends.

3.1 Layer 1 — retrieved exemplars + few-shot (runs today)

Given a prompt, we embed it, pull the k exemplars nearest it from the index, and lay them in front of a *frozen* model with a simple instruction: *here is how David writes; answer in that voice; imitate the manner, not the content*. This is the workhorse and the honest baseline. Telling a model “write like David” is weak; showing it the three paragraphs of mine most relevant to *this exact topic* is strong, because it anchors the model on my subject and my manner at once.

Why “manner, not content.” A frozen model reproduces surface voice well but will cheerfully *confabulate* opinions I never expressed — this is documented and unavoidable without retrieval-grounded stance modelling. So the system instruction explicitly forbids inventing my views, and the evaluation measures *style*, never belief.

3.2 Layer 2 — a StyleVector-style nudge (scaffolded)

There is a lovely result (StyleVector) that a person’s style is roughly a single *direction* in a model’s activation space, and that you can steer a frozen model along it with no training. The catch is that you must reach inside the model, and a closed API will not let you. The version I can run honestly today is therefore a *described* direction: I write down the axes where my voice leans — analogy first, explain the *why*, vary the sentence rhythm — and inject that description into the prompt. The genuine activation-space method is left as a clearly marked TODO that needs a small open model with hidden-state access; the steering equation it would implement is

$$\mathbf{h}' = \mathbf{h} + \alpha \mathbf{s}, \quad \mathbf{s} = \bar{\mathbf{h}}_{\text{authentic}} - \bar{\mathbf{h}}_{\text{style-agnostic}},$$

i.e. add a scaled style vector $\mathbf{s}$ (the contrast between activations on authentic vs. style-agnostic completions) to the residual stream.

3.3 Layer 3 — Horikawa-style refinement (scaffolded and gated)

This is the ambitious part. Horikawa’s “mind captioning” evolves a sentence so that its meaning-vector matches a target *decoded from a brain*. The transfer is almost too clean: replace “brain-decoded target” with “my style centroid” and you get a loop that repeatedly **masks** a few words, asks a masked language model to **propose** natural fillers, and **keeps** whichever candidate’s style embedding is closest to my centroid — while a length penalty stops the search from cheating by collapsing to something trivially short.

I implemented the control flow and, importantly, the *fitness function*, which I can compute right now because it is just cosine-to-centroid times a length penalty:

$$\text{fitness}(x) = \underbrace{\cos(\mathbf{v}_x, \mathbf{c})}_{\text{closeness to my voice}} \times \underbrace{\left(\frac{\ell}{\ell_0}\right)^{-\alpha}}_{\text{length penalty } (\alpha \approx 0.1)},$$

where ℓ is the candidate’s word count and ℓ_0 the target length. The one missing piece is the masked-LM proposer, which needs RoBERTa-large locally — a heavy download I have not approved — so the loop is `enabled=False` by default. If you turn it on, it reports exactly what it would do and changes nothing; it is non-destructive by construction (it only ever returns a *new* string).

```
python generate.py "Explain entropy to a beginner." # DRY RUN (no API calls)
python generate.py "Explain entropy to a beginner." --run # PAID: actually generate
python generate.py "..." --run --steer --refine # Layer 2 + (gated) Layer 3
```

4 Evaluation: three honest measurements

A model that *feels* like me on one example tells me nothing. The evaluator scores a candidate three ways, and the three together are far harder to fool than any one alone.

1) Neural cosine (primary). Cosine of the candidate’s embedding to my centroid, in the spirit of LUAR / StyleDistance. We also report the cosine to the *single nearest* exemplar, because a bland, generic output can drift toward the bland average and score deceptively well; it is much harder to also sit close to a real, specific passage of mine.

2) Interpretable stylometry (the dashboard I actually read). We profile my exemplars on a small, robust, dependency-free feature set — function-word rates (pronouns, articles, prepositions: the famously involuntary, topic-independent tells), sentence-length mean and variance (rhythm and burstiness), punctuation habits, vocabulary richness, mean word length — and report, per feature, a z -score: how many standard deviations the candidate sits from my range,

$$z_f = \frac{x_f - \mu_f}{\sigma_f}.$$

The output reads like a teacher’s margin notes: “*sentences too long* ($z = +1.2$), *too few first-person pronouns* ($z = -2.4$).” Embeddings can tell you *that* something is off; only this tells you *what*, and it is the tripwire against a candidate that games the embedding.

3) Authorship-attribution discriminator (the adversarial judge). The strongest honest test of a clone is whether a classifier can separate it from the real thing. The proper version trains a small model on my writing versus the clone’s and reports how easily it separates them — closer to a coin flip is better. That needs scikit-learn, which is not yet in the environment, so for now there is a transparent nearest-centroid fallback that returns a “clone-probability,” and the trained classifier is a clearly marked TODO.

```
python evaluate.py # FREE: sanity-check on a real exemplar
python evaluate.py "some generated text" # FREE: stylometry + discriminator
python evaluate.py "..." --neural # PAID: also embed for the cosine metric
```

5 Why this is honest about its limits

Two failure modes are worth saying out loud, because both are documented in the research.

Embedding metrics can be gamed. You can land near the centroid while sounding nothing like me. That is precisely why the stylometry dashboard exists as a second, independent check.

Interpretable features can also be gamed. You can hit my function-word rates while butchering the syntax. That is why the embedding cosine and (eventually) the discriminator sit beside the dashboard. No single number is trustworthy; the three together are much harder to fool, and a human who knows my writing remains the final judge.

Cost and safety. Every model call routes through `lib.costlog`, so it lands in `step_7/costs/usage.csv` like any archival run. The default runs are *free*: `data_prep.py` without `-embed`, `generate.py` without `-run`, and `evaluate.py` without `-neural` make zero API calls. Everything is written under `Style-Mimickry/data/`; my source writings are only ever read. No training, no fine-tuning, and no large downloads happen anywhere.

6 Caveats

- **Confabulated opinions are the real danger.** The model can invent stances I never held. The guardrail is the “manner, not content” instruction plus the fact that the evaluation only scores style. Faithful *opinions* would be a different, harder system.
- **Short text is unreliable.** Count-based features wobble on short inputs; judge on paragraph-sized outputs and aggregate over several.
- **The corpus is small for now.** Three explainers is enough to prototype, not to capture the full range of my voice; the fix is to point `CORPUS_DIRS` at more of my writing.
- **The neuroscience analogy has a seam.** Horikawa optimizes toward a *measured* neural target; here the target is a corpus-derived embedding, itself a lossy proxy for “my voice.” Promising, but a final polish, not the foundation.

Summary. The toolkit takes one idea seriously — if I cannot put my voice in the weights, I will carry it in the prompt and measure it from the outside — and follows it all the way down: real exemplars retrieved per request, a centroid that is the center of mass of my prose, and an evaluation that reports both *how* David-like and *which dimensions* are off. The advanced layers (StyleVector steering, Horikawa refinement) are scaffolded so the research path is visible, but gated so nothing heavy or paid runs until I say so.

Sources

- `Style-Mimickry-Research/style-mimickry-report.md` — architecture report (STRAP, LUAR, StyleDistance, StyleVector, ASTRAPOP, Horikawa mind-captioning, proxy-tuning).
- `Style-Mimickry-Research/LICW-incorporation.md` — interpretable-stylometry report (LIWC-22, Writeprints, MTLT, the composite-objective + dashboard design).
- Reused infrastructure: `pipeline_v3/step_7` — `config`, `lib/costlog`, `lib/providers/embed`, `lib/providers/llm`, `lib/vectorstore`.